

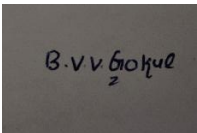
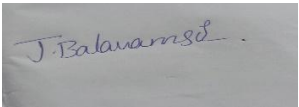
Annexure3b- Complete filing

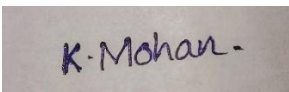
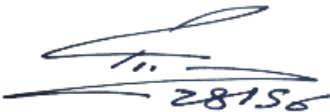
INVENTION DISCLOSURE FORM

Details of Invention for better understanding:

1. TITLE: Auto-Adaptive Real-Time Data Cleaning & Feature Engineering Framework for Edge-Enabled to IOT Systems.

2. INTERNAL INVENTOR(S)/ STUDENT(S):

A.	Full name	Bodangi Veera Venkata Gokul.
	Mobile Number	8978865468
	Email (personal)	bodhangigokul@gamil.com
	UID/Registration number	12219200
	Address of Internal Inventors	School Of Computer Science and Engineering Lovely Professional University, Punjab-144411, India
	Signature (Mandatory)	
B.	Full name	Bala Vamsi
	Mobile Number	8333871388
	Email (personal)	balavamsijujjavarapu@gmail.com
	UID/Registration number	12207318
	Address of Internal Inventors	School Of Computer Science and Engineering Lovely Professional University, Punjab-144411, India
	Signature (Mandatory)	

C.	Full name	Mohan Korra
	Mobile Number	9392847423
	Email (personal)	Mohankorra46@gmail.com
	UID/Registration number	12207117
	Address of Internal Inventors	School Of Computer Science and Engineering Lovely Professional University, Punjab-144411, India
	Signature (Mandatory)	
D.	Full name	Dr. Puneet Thapar
	Mobile Number	9501717653
	Email (personal)	puneet.thapar90@gmail.com
	UID/Registration number	28156
	Address of Internal Inventors	School Of Computer Science and Engineering Lovely Professional University, Punjab-144411, India
	Signature (Mandatory)	
E.	Full name	Dr. Shilpa Sharma
	Mobile Number	7009064460
	Email (personal)	shilpa13891@gmail.com
	UID/Registration number	13891
	Address of Internal Inventors	School Of Computer Science and Engineering Lovely Professional University, Punjab-144411, India

Signature (Mandatory)	
-----------------------	---

3. DESCRIPTION OF THE INVENTION:

1. Purpose

Aiming to transform data preprocessing inside Edge-Enabled IoT Systems, the Auto- Adaptive Real-Time Data Cleaning & Feature Engineering Framework It offers a smart, automated solution for straight-time generation of pertinent features directly on edge devices and cleaning of raw sensor data. Particularly when low latency and resource efficiency are critical for edge applications like predictive maintenance, real-time monitoring, and control systems, this invention is vital as the volume, velocity, and variability of IoT data overwhelm conventional static, cloud-based preprocessing methods.

2. Technical Workings

3. **Real-Time Data Ingestion & Profiling:** From many IoT sensors at the edge, the framework absorbs streaming data. As data comes in, it does first real-time profiling to grasp data properties, distributions, and possible quality problems.
4. **Adaptive Data Cleaning Engine:** This central unit utilizes a set of algorithms to manage typical data quality problems in real-time. This incorporates:
 - Employing statistical techniques (e.g., Z-score, IQR) or light ML models that learn dynamic thresholds from changing data patterns.
 - Using adaptive approaches (e.g., forward/backward fill, mean/median imputation based on recent windows, simple predictive models) selected according to data type and missingness patterns.
5. **Edge Optimization & Deployment:** The whole framework is optimized for resource-limited edge devices. Algorithms are chosen for computational effectiveness (low CPU/memory overhead). It provides mechanisms for state management efficiently and potentially offloading computationally expensive adaptation logic at intervals if necessary.
6. **Adaptive Feature Engineering Engine:** After cleaning, the engine generates useful feature appropriate for downstream edge analytics or AI models automatically. Methods include:
 - Computing rolling statistics (mean, max, min, variance) over adaptive time windows.
 - Using applicable transformations (e.g., scaling, normalization, log transforms) depending on the distribution of the data.
7. **Control & Monitoring Interface:** Offers mechanisms for setting up the framework (e.g., establishing initial parameters, specifying goals) and observing its performance in real-time.
8. **Unique Attributes**

1. **Auto-Adaptivity:** In contrast to static preprocessing pipelines, this approach adapts its cleaning engineering approaches dynamically according to shifting data properties (data drift) and patterns without human reconfiguration.
2. **Real-Time Edge Processing:** Undertakes sophisticated data cleaning and feature engineering on the edge device in real-time, reducing latency caused by transmitting raw data to the cloud and supporting local decision-making with speed.
3. **Integrated Cleaning & Feature Engineering:** Offers a single pipeline that processes both key preprocessing steps in a cohesive manner, ensuring features are derived from cleaned, trustworthy data.
4. **Resource Efficiency:** Optimized for use in edge environments, using lightweight algorithms and maximized processing to run within limited CPU, memory, and power budgets.
5. **Enhanced Data Quality for Edge AI:** Provides more quality, analysis-ready data and features to edge AI models or analytics apps in a manner that boosts their accuracy and reliability.

9. Conclusion

Auto-Adaptive Real-Time Data Cleaning & Feature Engineering Framework is a major leap forward for Edge-Enabled IoT systems. Auto-adapting and automating key data preprocessing steps in situ at the data source, this technology improves the timeliness, quality, and usefulness of IoT data, making it possible to build more robust, efficient, and smart edge applications.

A. PROBLEM ADDRESSED BY THE INVENTION:

The expansion of IoT devices creates massive amounts of raw data at the network edge. However, such data are noisy, incomplete, inconsistent, and not analysable or usable by machine learning models in a direct manner. This invention solves the following key problems:

1. **Poor Data Quality:** Raw IoT sensor data usually has missing values, outliers, noise, and calibration errors, which result in incorrect analytics and less-than-optimal performance of AI models running at the edge.
2. **Latency of Cloud Processing:** Outsourcing huge volumes of raw IoT data to the cloud for scrubbing and feature engineering brings in significant latency, which is intolerable for real-time applications such as industrial control, autonomous systems, or emergency response.
3. **Manual and Static Preprocessing:** Conventional data processing systems are usually developed through manual work and are static. Developing and sustaining such systems are resource-intensive, and they do not learn to accommodate changes in data features or changes in underlying process (data drift), ultimately causing performance to degrade over time.

4. **Edge Device Resource Limitations:** Edge devices, such as gateways, microcontrollers, or embedded systems, have limited computing power, memory, and energy resources, thus creating challenges in running advanced data processing and algorithms on these devices.
5. **Feature Engineering Bottleneck:** It takes time, is domain expert-sensitive, and challenging to perform well in real-time at the edge to manually discover and engineer pertinent features from raw, high-rate sensor data.
6. **Ineffective Bandwidth Utilization:** The movement of large amounts of low-quality raw data to cloud infrastructure demands high network bandwidth and incurs associated costs.

7. Conclusion

The Auto-Adaptive Real-Time Data Cleaning & Feature Engineering Framework addresses these challenges by offering an autonomous, intelligent, edge-native solution.

B. OBJECTIVE OF THE INVENTION (Provide minimum two)

- ☐ **Automate Real-Time Edge Data Preprocessing:** To create a framework that automatically conducts data cleaning (dealing with missing values, outliers, noise) and feature engineering (generating informative variables) on streaming IoT data in real-time on edge devices with minimal manual intervention.
- ☐ **Make Adaptive Data Pipelines:** To build a system whose data cleaning and feature engineering methods automatically adjust to evolving data distribution, patterns, and quality (data drift) over time to maintain optimal performance without the need for manual adjustments.
- ☐ **Optimize for Resource-Limited Environments:** To implement the framework with computationally efficient algorithms and data structures to deploy on resource-constrained edge devices with limited processing, memory, and energy budgets.
- ☐ **Enhance Quality and Utility of Edge Data:** To increase the reliability and informational value of data at the edge, thus enhancing the accuracy, robustness, and effectiveness of local analytics, control systems, and AI/ML models.

C. STATE OF THE ART/ RESEARCH GAP/NOVELTY:

Sr. No.	Patent Name and Patent ID's	Abstract	Research Gap	Novelty
1	Systems and methods for data cleaning (US9875382)	Concerned with methods for detecting and fixing errors (outliers, inconsistencies, missing values) in datasets, typically offline or in cloud environments.	Typically, do not have real-time processing capabilities, are statically (non-adaptive) operating, and are not particularly optimized for resource-limited edge devices. Typically, only clean	Combines real-time, adaptive cleaning specifically tailored to edge constraints within a broader framework.
2	Real-time-stream processing frameworks_ (e.g., Apache Flink, Spark Streaming related patents)	Offer general platforms to process data streams, supporting tasks such as filtering, aggregation, and transformations.	Mainly generic platforms needing custom logic for cleaning/FE. Adaptation logic usually user-coded and sophisticated. Often heavyweight for common edge devices.	Offers a special application-specific framework for adaptive cleaning and feature engineering, optimized for deployment on the edge, not a generic stream processor.
3	Automated feature engineering systems (US10599811)	Explains systems that create candidate features automatically from raw data, usually for machine learning applications, normally executed in cloud/server environments.	Usually run offline or in batch mode, highly computationally intensive, not optimized for real-time edge deployment, and do not normally include adaptive cleaning.	Merges real-time, adaptive feature engineering with adaptive cleaning, particularly designed to thrive with edge resource limitations. The data drift-based adaptivity is the

				main differentiator
4	Edge computing data management (US10783102)	Aims to manage edge storage, filtering, and low-level processing to minimize data transfer to the cloud.	Simple filtering/aggregation is prevalent, but does not support advanced, adaptive cleaning and high-end feature engineering capabilities specific to downstream tasks.	Excels beyond basic filtering to adopt an end-to-end, auto-adaptive preprocessing pipeline (both cleaning and FE) in real-time at the edge.
5	Adaptive systems for sensor networks (Related academic works/patents)	Investigates how sampling rates, routing, or simple filtering can be adapted with respect to network conditions or data variations.	Adaptation usually only applies to network layers or very straightforward data parameters, not sophisticated, end-to-end data cleaning and feature engineering pipelines.	Applies adaptation directly to the data preprocessing pipeline (cleaning techniques, parameters, feature selection) upon real-time data attributes and possibly model feedback, run efficiently at the edge.

Conclusion

The Auto-Adaptive Real-Time Data Cleaning & Feature Engineering Framework addresses pivotal research gaps through a holistic, edge-native system that integrates data cleaning and feature engineering, achieves these operations in real-time, and most importantly, auto-adapts the whole process according to evolving data dynamics. It overcomes limitations of current static, cloud-nativized, or single-element approaches, unlocking more resilient and smarter edge-enabled IoT systems.

D. DETAILED DESCRIPTION:

Auto-Adaptive Real-Time Data Cleaning & Feature Engineering Framework for Edge-Enabled IoT Systems is a new and novel software architecture that can run efficiently on edge devices. It offers an end-to-end pipeline for processing raw, mostly untrustworthy, sensor data to clean, insightful features that can be used in real-time analytics and decision-making.

- **2. System Components**

- 2.1 Data Ingestion Interface:**

- Connects to various data sources at the edge (e.g., sensors via MQTT, CoAP, local data streams).
 - Handles different data formats and protocols.
 - Buffers incoming data for stream processing.

- 2.2 Real-Time Data Profiler:**

- Continuously calculates lightweight statistics on incoming data streams (e.g., min, max, mean, variance, count of missing values, estimated distribution parameters) over sliding windows.
 - Detects abrupt changes or drifts in data characteristics.
 - Provides inputs to the adaptive engines.

- 2.3 Adaptive Cleaning Engine:**

- Anomaly Detection Module: Applies algorithms such as adaptive Z-score, IQR, or lightweight clustering (e.g., incremental k-means) to detect outliers. Model parameters or thresholds dynamically adjust based on profiled statistics of the data. Anomalies can be flagged, trimmed, or capped.
 - Missing Value Imputation Module: Chooses and executes imputation methods according to data type, missingness proportion, and real-time properties. Available choices are adaptive mean/median/mode imputation, LOCF/NOCB (Last Observation Carried Forward/Next Observation Carried Backward), or basic time-series forecasting models (e.g., ARIMA, Exponential Smoothing) with parameters updated from time to time.
 - Noise Reduction Module: Uses filters such as adaptive moving averages, Savitsky-Golay filters, or Kalman filters. Filter parameters (e.g., window, smoothing factors) are varied based on noise level estimates from the profiler.

- 2.4 Adaptive Feature Engineering Engine:**

- Temporal Feature Generator: Generates features such as rolling averages, sums, standard deviations, min/max values over dynamically varying time windows. Window sizes are varied based on data velocity or observed seasonality/periodicity.
- Transformation Module: Scales (e.g., Min-Max, Standardization), normalizes, or applies power transforms according to profiled data distributions to stabilize variance or satisfy model assumptions.
- Interaction & Domain-Specific Feature Module: Produces polynomial features, ratios, or differences between sensor streams. Can include pre-defined domain rules or apply lightweight feature selection methods (e.g., correlation, simple model importance) to adjust the feature set.

2.5 Adaptation Controller & State Manager

- Coordinates the adaptation process based on feedback from the Profiler and possibly from downstream models/applications.
- Picks suitable algorithms and optimizes their parameters for the Cleaning and Feature Engineering engines.
- Handles state required for adaptive algorithms (e.g., up-to-date statistical estimates, filter states).
- Handles transitions when strategies change smoothly.

2.6 Edge Resource Monitor:

- Tracks CPU, memory, and energy usage of the framework components.
- Returns feedback to the Adaptation Controller to keep operations within device limits (e.g., reducing algorithms in heavy load).

2.7 Output Interface:

- Streams the sanitized data and engineered features to local applications (e.g., edge AI models, control loops, dashboards) or sends the compressed, higher-value data to the cloud.

3. Technical Functionality

3.1 Real-Time Stream Processing:

- Processes data in real time as it becomes available with near-zero latency through efficient stream processing.

3.2 Dynamic Algorithm Selection and Parameter Tuning:

- The Adaptation Controller dynamically selects the most appropriate cleaning/FE algorithms and fine-tunes their parameters (e.g., outlier thresholds, imputation strategies, filter coefficients, aggregation windows) according to real-time data profiling and resource restrictions.

3.3 Optional Feedback Loop:

- May add performance feedback from downstream edge AI models (e.g., accuracy of predictions, error rates) to further direct the adaptation of feature engineering tactics.

3.4 Lightweight Implementation:

- Leverages algorithms and data structures designed for minimal computational overhead and memory usage appropriate for common edge hardware.

3.5 Configurability:

- Enables users to specify initial settings, constraints, adaptation policies, and objectives for the preprocessing pipeline.

4. Unique Features

4.1 Fully Automated Pipeline:

- Combines profiling, cleansing, feature engineering, and adaptation into an end-to-end, automated flow at the edge.

4.2 Enhanced User Experience:

- Adjusts not only parameters but perhaps the selection of algorithms employed in cleaning and feature engineering depending on data context and available resources.

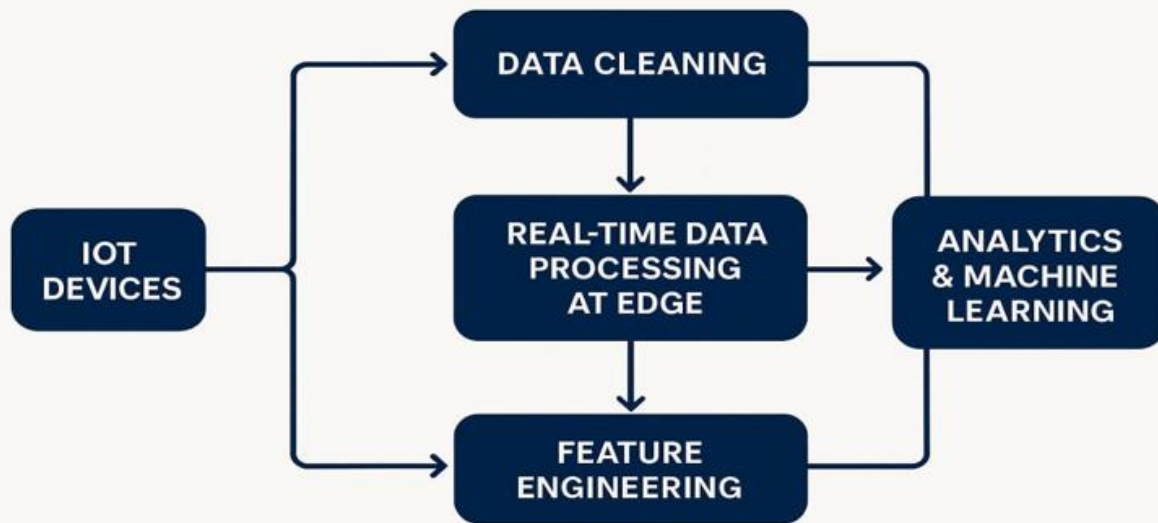
4.3 Robustness against Data Drift:

- Designed to cope with concept drift and patterns of data variation typical of the IoT domain, ensuring data quality and feature applicability in the long term.

Conclusion

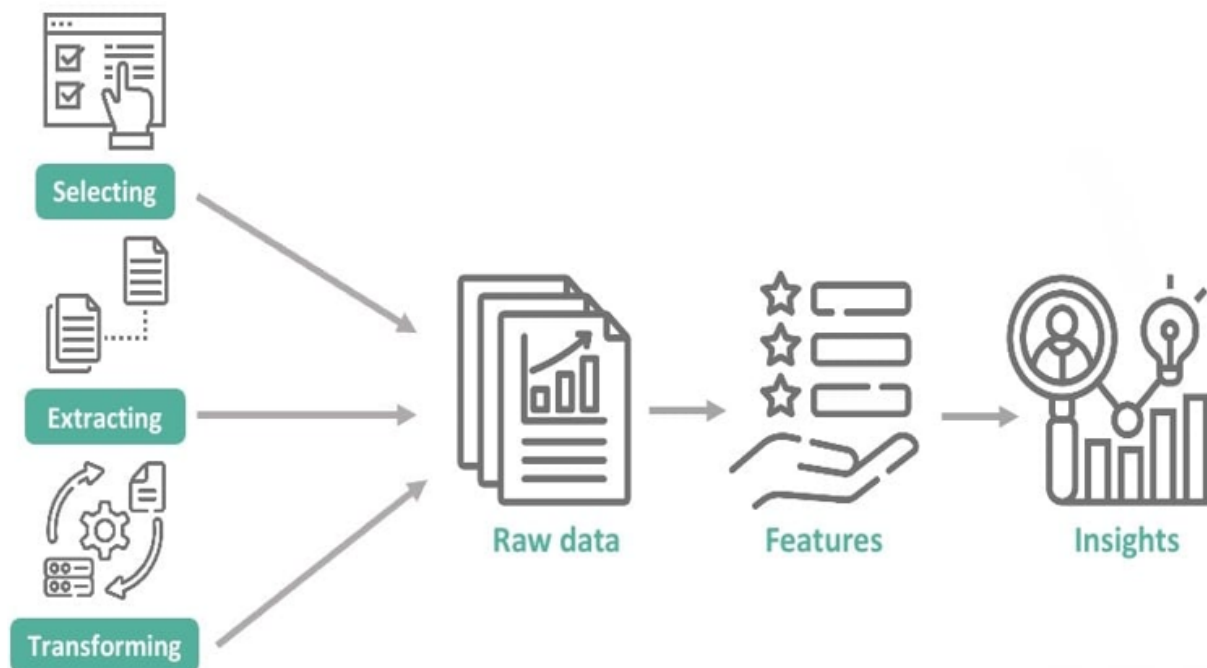
The architecture in detail emphasizes a highly advanced yet edge-optimized design. Through the combination of real-time profiling with adaptive cleaning and feature engineering engines, controlled by a resource-aware controller, this system offers a strong, self-sustaining solution for optimizing the value of data produced within Edge-Enabled IoT Systems.

Process Workflow:



Real-Time Data Cleaning & Feature Engineering Framework For Edge-Enabled IoT Systems

What is Feature Engineering?



E. RESULTS AND ADVANTAGES:

- Auto-adaptive real-time data cleaning and feature engineering framework distributes many consequences and benefits, which makes it a better solution than the current pre-art for the edge IOT system:
- 1. Operations efficiency and low delay:
 - Real-Time Insights: By executing data processing on the edge, the system essentially reduces the delay between the original and actionable insight or model predictions, required in the system time-intelligent applications.
 - Automation: Tightening and error-incorporated data automate cleaning and feature engineering, issues data to scientists and engineers and reduces operating overheads.
- 2. Better data quality and model performance
 - Increased data reliability: Produces clean, consistent data by managing frequent noise, out lair and missing values, resulting in more reliable analytics.
 - Better AI/ML accuracy: AI supplies high quality, relevant features to sideline models, making them much better in predicting results, resistant to noise and making it very much better.
 - Data drift flexibility: The adaptive character ensures that the preprosaning of the data as the underlying data pattern shift will continue to work well, the adaptive character ensures that the decline in the performance of downstream applications over time.
- 3. Customized Resource Uses:
 - At least bandwidth uses by purifying the data and filtering valuable features on the edge, only the necessary information should be sent to the cloud (if necessary), to reduce the network bandwidth use and expenditure a lot.
- **Effective Edge Processing:** With lightweight algorithms and resource control, the system is effective within the computation and energy limitations of edge devices.
- 4. Scalability and flexibility
 - **Distributed processing:** Originally distributed is integrated with the IOT framework, locally processing on several edge equipment.
 - **Flexibility to support heterogeneous data:** The adaptive nature of the framework enables it to support several sensor data types and to separate environmental conditions without continuous manual adjustment for every situation.

- **5. Low development and maintenance attempt:** • Provided pipeline development: Custom for every edge application provides an easy-to-up-planned outline to reduce the effort to develop custom preposing pipelines.
- **Low maintenance:** The auto-adaptability feature reduces the need for continuous manuals and maintenance of the preprosaucating logic with changing data.
- **Comparison of existing pre-art • Vs. Cloud-based ETL/Preprocessing:** Gives quite low delay and low bandwidth cost. Network provides tolerance for intervention.
- **vs. Static Edge Filter:** Extending too much developed, challenging cleaning and engineering operations favorably, not only stable filtering rules.
- **Vs. Hand Preprocessing:** Full, real-time, and adaptive automatic processing, removing human-in-loop bottlenecks and allowing stability.
- **vs. Standalone Cleaning/FE Equipment:** Both stages connect unattained within an edge-oriented, adaptive system.

Conclusion

Framework provides tangible benefits in terms of speed, data quality, resource efficiency and automation. Its ability to customize directly in real time on the edge creates a better and required solution to unlock the full potential of data in the modern IOT system compared to traditional, stable or cloud-ritual approaches.

F. Expansion:

- To guarantee complete coverage and successful deployment of auto-adaptive real-time data cleaning and feature engineering framework, some important variables are required to be considered. These variables affect the design, signs, setups and overall success of the system:
 - **1. Edge device capabilities:**
 - **Hardware Resources:** CPU clock speed, number of cores, amount of RAM, storage space, and power budget (battery or main) largely determine the complexity of algorithms that can work successfully
 - **Operating systems and software environment:** support for the presence of various age OS (Linux, RTO variants) and the presence of essential libraries (eg, python, C ++ libraries).
 - **2. Data characteristics:**

- **Sensor types and data modalities:** Supporting various data such as numerical time chain, classified data, GPS coordinates, images (if suitable for metadata extraction).

- **Data volume and velocity:** The rate and quantity of incoming data determines the processing burden and buffer size.

- **Data Structure and Format:** Structured (eg, JSON, CSV) vs. Management of unnecessary data currents.

- **Nature of noise and discrepancies:** The nature and frequencies of anticipated data quality problems determined the selection of early cleaning algorithms.

- **3. Application requirements:**

- **Loastiness Obstacles:** Maximum delay for data processing data for target applications (eg, real -time control vs. hourly reporting).

- **Downstream model is needed:** Edge AI model or accurate features required by analytics work.

- **Accuracy vs. Resource Business Governing:** Desirable trade-closes between preprosacing completion and resource use.

- **Response Availability:** Is available from the downstream application available to notify the performance reaction optimization.

4. Adaptation Strategy: • Adaptation trigger: How changes are sensory (e.g., statistical drift detection mechanisms, time-based triggers, external commands).

- **Adaptation scope:** What is customized (parameter, algorithm, feature set).

- **Adaptation speed versus stability:** How fast the system prevents excessive oscillations.

- **Learning Mechanism:** How the system receives the best configuration (e.g., reinforcement learning, baysian adaptation - possibly offline learning that feeds in online adaptation).

- **5. Deployment and Management:**

- **Framework configuration:** Ways to configure early parameters, rules and obstacles.

- **Monitoring and logging:** facilities for performing performance, resource usage and decisions taken by adaptive engines.

- **Updates and maintenance:** procedures to update framework software in edge equipment.

- **Security:** Protecting data privacy and safe communication, especially if interacting with clouds or other systems.

- **6. network status:**

- **Connectivity:** Conversation with cloud elements requires network connection availability and stability (eg, computation offloading for model updates).
- **bandwidth:** The available bandwidth affects how to send raw vs processed data.

- **Conclusion**

For effective deployment of auto-adaptive framework, it is necessary to handle these variables in design, purinogen and configuration. Complete consideration allows the framework to adjust the special edge environment, data features, and application requirements, which adapt to its efficiency, reliability and effectiveness in a wide range of IOT applications.

G. EXISTING DATA:

- To provide adequate convenience to support, prepare and verify the auto-acadeeptive real-time data cleaning and feature engineering framework, one must refer to existing data sources, benchmarks and comparative tasks. Some related categories of data include:

- **1. Public IOT Sensor Dataset:**

- **Source Example:** Intel Berkeley Research Lab Dataset (sensor for environmental monitoring), UCI machine learning repository (sensor dataset like gas sensors, domestic power consumption), dataset focus on future maintenance, smart cities or environmental monitoring from sites such as Gas Sensors, Domestic Power Consumption).

- **Utility:** Offer realistic raw data currents with built -in noise, missing values and potential drifts to challenge the cleaning and optimization of framework under various circumstances.

- **2. Fake data:**

- **Method:** Creating artificial time-series data for which the ground truth is known, and problems are (e.g., various noise types, prescribed drift patterns, various missing models) can be offered in a controlled fashion.

- **Utility:** To enable controlled experiment to correctly assess the quality of cleaning algorithms, facilities engineering methods and quality of adaptation policies under specified conditions.

- **3. Data cleaning algorithm benchmark**

- **Source Example:** Academic papers and comparative studies that evaluate outlier detection methods, copying techniques, (eg, compared to KNN vs. KNN vs. model-model-based

copy of KNN vs. KNN vs. KNN vs. KNN vs. KNN vs. KNN vs.), and noise reduction filters.

- **Utility:** Provide performance baselines for individual components of the cleaning engine and help in selecting appropriate initial algorithms.
- **4. Benchmark for feature engineering techniques:**
- **Source Example:** Studies that evaluate the efficacy of different feature engineering methods (e.g., temporary aggregation, changes) on tasks such as time-serial prediction or classification with sensor data. Automatic feature engineering (autopsy) tool comparison.
- **Utility:** Assist in verification of utility in facilities created and adaptive selection process.
- **5. Edge device display profile**
- **Source Example:** Benchmarking results for typical age CPU/MCUS (e.g., Raspberry Pie, Jetson Nano, ESP 32), performance characteristics of light-wisdom ML libraries (e.g., TensorFlow Light, ONNX runtime), measurement of power consumption.
- **Utility:** Supply of information to notify selection between computationally viable algorithms and to establish realistic resource limits for resource monitor components.
- **6. Real world pilot project data (if applicable):**
- **Source:** Target Edge Hardware and Data collected from initial deployment or test band on sensors.
- **Utility:** Special application provides the most applied information to correct the outline of domains and hardware.
- **Conclusion**

Using these various existing data sources is necessary for recurring growth, complete testing and performance verification of auto-adaptive framework. Comparative probes and benchmarks aid in proving benefits on pre-art, while realistic and fake dataset guarantee that framework is strong and effective in practical age IOT use cases. The setup initial test may employ public dataset (e.g., UCI or Kagel) and appoint simulated data for controlled experimental evaluation

.4. USE AND DISCLOSURE (IMPORTANT): Please answer the following questions:

A. Have you described or shown your invention/ design to anyone or in any conference?		NO (No)
---	--	---------

B. Have you made any attempts to commercialize your invention (for example, have you approached any companies about purchasing or manufacturing your invention)?		NO (No)
C. Has your invention been described in any printed publication, or any other form of media, such as the Internet?		NO (No)
D. Do you have any collaboration with any other institute or organization on the same? Provide name and other details.		NO (No)
E. Name of Regulatory body or any other approvals if required.		NO (No)

5. Provide links and dates for such actions if the information has been made public (Google, research papers, YouTube videos, etc.) before sharing with us. **NA**

6. Provide the terms and conditions of the MOU also if the work is done in collaboration within or outside university (Any Industry, other Universities, or any other entity). **NA**

7. Potential Chances of Commercialization. **Yes**

8. List of companies which can be contacted for commercialization along with the website link.

Following are the IoT platform, edge computing, and data analytics companies that might be a potential commercialization partner for them:

1. AWS (Amazon Web Services)

- **Description:** Provides AWS IoT Core, AWS IoT Greengrass (for edge computing), and comprehensive data analytics. Their framework could extend to their edge offerings.
- **Website:** [AWS IoT – Industrial, Consumer, Commercial, Automotive – AWS](#)

2. Microsoft Azure

- **Description:** Offers Azure IoT Hub, Azure IoT Edge, and Azure Machine Learning. This framework would be a great module for their edge AI solutions.
- **Website:** [Azure IoT – Internet of Things Platform | Microsoft Azure](#)

3. Google Cloud Platform (GCP)

- **Overview:** Provides Google Cloud IoT Platform and Edge TPU. Incorporating this framework would enhance data quality for their edge AI hardware and cloud services.
- Website: <https://cloud.google.com/iot>

9. Any fundamental patent which has been utilized, and we must give royalty to them.

None have been identified yet. The system seeks to bring in and modify existing techniques, but the innovation is in the combination, adaptation process, and edge optimization. Existing open-source libraries may be utilized, provided that they are under their respective licenses (e.g., Apache, MIT), which generally do not involve royalty fees.

10. FILING OPTIONS: Please indicate the level of your work which can be considered for provisional/ complete/ PCT filings – (Provisional)

11. KEYWORDS:

- **Edge Computing**
- **IoT (Internet of Things)**
- **Real-Time Data Processing**
- **Data Cleaning**
- **Feature Engineering**
- **Adaptive Systems**
- **Machine Learning**
- **MLOps (Machine Learning Operations)**
- **Data Preprocessing**
- **Anomaly Detection**
- **Missing Value Imputation**
- **Noise Reduction**
- **Edge AI**
- **Data Drift**

- **Sensor Data**
- **Automated Data Pipeline**
- **Resource Constrained Devices**
- **Stream Processing**
- **Intelligent Edge**

(Letter Head of the external organization)

NO OBJECTION CERTIFICATE

This is to certify that University/Organization Name, or its associates shall have no objection if Lovely Professional University files an IPR (Patent/Copyright/Design/any other.....) entitled "....." including the name(s) of,.....as inventors who is(are) student(s)/employee(s) studying/ working in our University/ organization.

Further Name of the University/Organization shall not provide any financial assistance in respectof said IPR nor shall we raise any objection later with respect to filing or commercialization of the said IPR or otherwise claim any right to the patent/invention at any stage.

(Authorised Signatory)